

Análisis de la herramienta ESBMC

Efficient SMT-based Context-Bounded Model Checker

Tomás Achával, Mauricio Llugdar, Gregorio Vilaro

Historia de la herramienta

- Creada en 2008 por Lucas Cordeiro, Bernd Fischer y Joao Marques-Silva
 - Surgió como evolución de CBMC
 - Soporta C, C++, CUDA, ChERI, Kotlin, Python, Rust, y Solidity
 - Open Source
-

Objetivos de ESBMC

Verificar automáticamente la corrección del software y detectar vulnerabilidades lógicas o de seguridad antes del despliegue.

Lenguajes no memory-safe (C): punteros colgantes, desreferencia nula, desbordamiento de búfer, fugas de memoria.

Lenguajes memory-safe (Python): desbordamientos aritméticos, división por cero, aserciones del usuario, razonamiento orientado a objetos.



Fases del desarrollo donde es útil

Todas las que involucren código.

Propiedades que verifica

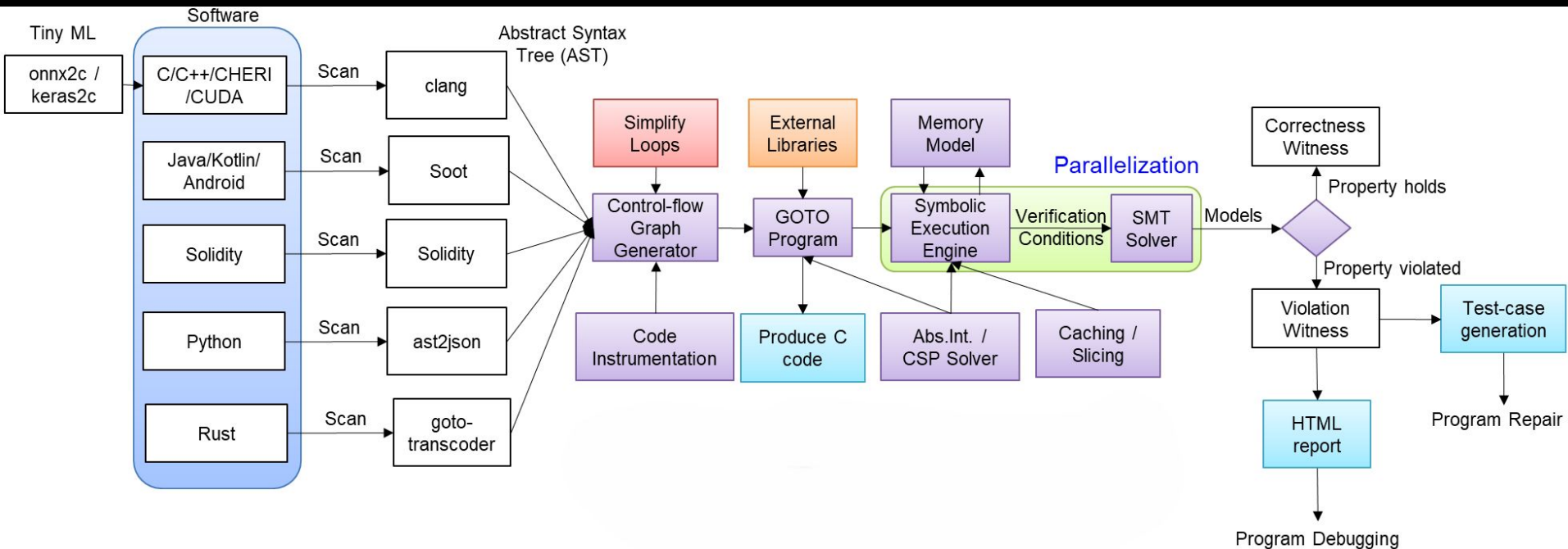
Safety y liveness: deadlocks, aserciones, propiedades definidas por el usuario, seguridad de memoria, desbordamientos aritméticos, ...

Salida ante un error

Contraejemplo que indica exactamente dónde y cómo ocurre el problema.



Arquitectura



Aspectos Técnicos

Sistema de transiciones (S, S₀, T)

ϕ : propiedad a verificar

$$\psi_k = I(s_0) \wedge \bigwedge_{i=0}^{k-1} T(s_i, s_{i+1}) \wedge \bigvee_{i=0}^k \neg \phi(s_i)$$

Desenrollado acotado

- ❖ No falsos positivos
- ❖ “Falsos negativos”

K-Inducción

- ❖ Puede introducir falsos positivos
- ❖ Si converge, no falsos negativos

$$\text{kind}(P, k) = \begin{cases} P \text{ contiene un bug,} & \text{si } B(k) \text{ es SAT} \\ P \text{ es correcto,} & \text{si } B(k) \wedge [F(k) \vee In(k)] \text{ es UNSAT} \\ \text{kind}(P, k + 1), & \text{en otro caso} \end{cases}$$

¿Cómo se utiliza?

Desde línea de comandos ‘`esbmc archivo <opciones>`’

- Método de verificación: `--k-induction`, `--unwind N`, `--incremental-bmc`
- Propiedades específicas: `--memory-leak-check`, `--data-races-check`, ...

Backends

- Integrados: Bitwuzla, Boolector, Z3, MathSAT, CVC4, Yices.
- Otros? `--smtlib --smt-formula-only --output (SMTLIB v2)`

Generación de tests

- `--branch-coverage --generate-ctest-testcase`

Anotaciones

- `__ESBMC_X(_)`, donde `X = assert, requires, ensures, assume, ...`
- `nondet_int()`, ...

Diferencias con CBMC

Para C/C++	ESBMC	CBMC
Frontend	Clang	Propio
Backend	SMT, seleccionable	Principalmente SAT
Técnicas	k-inducción integrada	k-inducción requiere otra herramienta
Concurrencia	Exploración <i>lazy</i> y <i>schedule-recording</i> Partial Order Reduction	Codificación de fórmula única

Casos de Estudio

Arm RMM (Realm Manager Monitor) (2024)

- ESBMC detectó 23 fallos nuevos no encontrados por CBMC.
 - Desventaja: en verificación multi-propiedad, ESBMC fue más lento (>5000 s) debido a llamadas repetidas a su SMT-solver.
- CBMC utiliza SAT incremental (MiniSat), resultando más rápido.



Casos de Estudio - Arm RMM

Ejemplo simple de una falla encontrada por ESBMC al ejecutar RMI_REALM_DESTROY

```
#include <assert.h>
int arr[8] = {1,2,3,4,5,6,7,8};
int main() {
    int *a = &arr[7];
    if((unsigned long)a >= (unsigned long)(-4095))
        assert((unsigned long)(-1*(long)a) < 6); //esbmc fails, cbmc success
}
```

ESBMC y CBMC difieren en la interpretación de casteos de punteros a enteros

$\&arr[7] = -2048$

$(\text{unsigned long})(-2048) > (\text{unsigned long})(-4095)$

$(\text{unsigned long})(-1 * (\text{long})a) = 2048 > 6$
ASSERT debería FALLAR

Más Casos de Estudio

ESBMC-Arduino

- Mejora la seguridad del código C en la plataforma Arduino.
- Aplicaciones: sistemas embebidos críticos, desarrollo basado en contratos, docencia e investigación en verificación formal.



ESBMC-Python

- Verificación del protocolo de consenso de Ethereum
- Interrupciones de servicio, ataques a la red, ...



Ethereum Consensus Specification

Handle `integer_squareroot` bound case #3600

Merged

[hwwhww](#) merged 3 commits into [dev](#) from [integer_squareroot](#) on Feb 15, 2024

Conversation 4 Commits 3 Checks 0 Files changed 5



[hwwhww](#) commented [on Feb 14, 2024](#) • edited ▾

Contributor ...

Credits to the University of Manchester Bounded Model Checking (BMC) project team: Bruno Farias, Youcheng Sun, and Lucas C. Cordeiro for reporting this issue! 🙌 100

This team is an [Ethereum Foundation ESP](#) "Bounded Model Checking for Verifying and Testing Ethereum Consensus Specifications (FY22-0751)" project grantee. They used [ESBMC model checker](#) to find this issue.

Also, thanks to Mate Soos and Justin Traglia ([@jtraglia](#)) for helping verify the issue! 🙌

tl;dr

It's a spec bug, but it's impossible to produce it in the current mainnet.

Código Original

Archivo int_sqrt.py:

```
def integer_squareroot (n : uint64 ) -> uint64 :  
    """  
    Return the largest integer ''x'' such that ''x **2 <= n''.  
    """  
    x = n  
    y = (x + 1) // 2  
    while y < x :  
        x = y  
        y = (x + n // x) // 2  
    return x
```

esbmc int_sqrt.py --function integer_squareroot --k-induction

[Counterexample]

State 1 file int_sqrt.py line 5 column 4 function integer_squareroot thread 0

x = 0xFFFFFFFFFFFFFFFF (11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111111)

State 2 file int_sqrt.py line 6 column 4 function integer_squareroot thread 0

y = 0 (00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000)

State 3 file int_sqrt.py line 8 column 8 function integer_squareroot thread 0

x = 0 (00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000)

State 4 file int_sqrt.py line 9 column 8 function integer_squareroot thread 0

Violated property:

file int_sqrt.py line 9 column 8 function integer_squareroot

division by zero

x != 0

VERIFICATION FAILED

Bug found (k = 1)

Código Corregido

```
def integer_squareroot (n : uint64 ) -> uint64 :  
    """  
    Return the largest integer ''x'' such that ''x **2 <= n''.  
    """  
    if n == UINT64_MAX:  
        return UINT64_MAX_SQRT  
    x = n  
    y = (x + 1) // 2  
    while y < x :  
        x = y  
        y = (x + n // x) // 2  
    return x
```

Conclusiones

- ★ Facilidad de uso
- ★ Cantidad de lenguajes soportados
- ★ Compatibilidad del back-end

- ❑ Buen pero mejorable desempeño en SV-COMP

- Falsos positivos en k-inducción
- Lentitud en chequeo multi-propiedad
- Incompletitud del soporte a cada lenguaje

¿Preguntas?